基于 2024 年 NYC 网约车数据的机器学习建模报告

姓名：陈潼欣　学号：25210690011

2026 年 1 月 11 日

摘要

本报告基于 NYC TLC HVFHV 2024 年海量网约车订单数据，利用机器学习方法解决了两个核心问题。第一部分针对**乘客打车开销与乘坐时间**构建了回归预测模型。通过“按月切分 + 哈希抽样 + 目标稳健变换”的策略，有效处理了亿级数据规模，并对比了 XGBoost、LightGBM 与 Ridge 回归的性能，证明了非线性模型在捕捉时空特征交互方面的优势。第二部分针对**城市拥堵程度**构建了综合建模方案。采用“无监督聚类 + 有监督预测”的两阶段策略：首先利用 K-Means 算法识别出“恶劣天气缓行”、“高需求拥堵”等四类典型交通模式；随后构建 Random Forest 分类器，基于时间与天气特征实现对未来交通拥堵状态的有效预警。实验结果表明，该方案不仅能够精准预测行程属性，还能从机理上区分并预测不同类型的城市拥堵。

关键词：出行预测；城市计算；K-Means 聚类；拥堵模式；梯度提升树

目录

1 第一部分：预测乘客打车开销与乘坐时间的模型 4
1.1 任务定义与业务背景 4
1.1.1 问题定义 4
1.1.2 目标与约束 4
1.2 数据来源、字段与数据字典 4
1.2.1 数据来源与文件组织 4
1.2.2 外部数据：天气与假期数据来源 4
1.2.3 原始数据字典 5
1.2.4 特征工程后数据字典（训练用 features parquet） 5
1.2.5 目标变量分布（长尾性与异常值） 7
1.2.6 关键特征分布与合理性检查 7
1.3 训练数据构造与预处理流程 10
1.3.1 时间切分：避免未来信息泄露 10
1.3.2 哈希抽样策略：2% → 5% 10
1.3.3 目标变量稳健变换：winsorization + log1p 11
1.3.4 缺失值与数据类型处理 11
1.4 特征工程设计 11
1.4.1 建模时点选择：以 `request_datetime` 为预测时点（避免信息泄露） 11
1.4.2 基础时空特征（时间周期性 + 空间差异） 12
1.4.3 行程长度特征（距离对费用与耗时的核心解释力） 12
1.4.4 平台与订单属性特征（派单策略与服务类型差异） 12
1.4.5 天气与假期特征（外部信息融合） 12
1.4.6 样本过滤与落盘（保证训练稳定与内存安全） 13
1.5 模型方法与训练策略 13
1.5.1 统一评估指标与还原方式 13
1.5.2 训练阶段策略：时间切分、抽样训练与稳健目标变换 13
1.5.3 XGBoost（hist） 14
1.5.4 LightGBM（原版 → 更快更稳版） 14
1.5.5 Ridge 回归（线性基线） 15
1.5.6 抽样比例与数据规模（2% vs 5%） 16
1.5.7 结果表 16
1.5.8 结果解读 17
1.6 小结与改进方向 17
1.6.1 小结 17
1.6.2 改进方向 18

2 第二部分：构建对城市拥堵程度进行建模的模型 19
2.1 问题定义与建模策略 19
2.2 数据处理与特征工程 19
2.2.1 数据清洗与预处理 19

2.2.2 时空聚合与特征构造 19
2.2.3 标准化处理 20
2.3 第一阶段：基于聚类的拥堵模式发现 20
2.3.1 结果解读 22
2.4 第二阶段：拥堵状态预测模型 22
2.4.1 模型构建与实验设置 22
2.4.2 实验结果与评估 22
2.4.3 拥堵成因与特征重要性分析 23
2.5 结论与业务价值 24

3 总结与展望 24

A 附录：关键代码实现 26
A.1 第一部分：特征工程与模型训练 (LightGBM) 26
A.2 第二部分：聚类与分类建模 26

1 第一部分：预测乘客打车开销与乘坐时间的模型

1.1 任务定义与业务背景

1.1.1 问题定义

给定订单请求阶段可得信息（请求时间、起终点区域、里程、平台/派单属性、共享/无障碍标记，以及天气/假期等外部因素），构建两个监督学习回归模型：

- **费用预测：** $y_{\text{cost}} = \text{y_cost_no_tips}$，表示不含小费的乘客开销；
- **时长预测：** $y_{\text{time}} = \text{y_time}$，表示行程时长（秒）。

1.1.2 目标与约束

- 目标：在不泄露未来信息的前提下，最大化对未来月份的预测精度（RMSE/MAE）。
- 约束：原始数据规模为数亿级别（月均 2000 万条，不能全量多轮训练；需通过抽样与高效 I/O 控制计算成本。

1.2 数据来源、字段与数据字典

1.2.1 数据来源与文件组织

- 数据来源：NYC TLC HVFHV 2024 月度 parquet 文件（`fhvhv_tripdata_2024-01.parquet` ∼ `...-12.parquet`）。
- 特征数据：`/data/tlc_hvfhv_2024_features_wxhol/features_2024-XX.parquet`（已完成天气 + 假期 join）。

1.2.2 外部数据：天气与假期数据来源

- 天气数据来源：Open-Meteo Historical Weather API（Archive API）。
  - 抓取方式：使用 `requests` 请求接口，解析为逐小时序列后写入 parquet（抓取脚本中使用 `pandas` 进行整理）。
  - 地理位置：纽约市（示例坐标 `lat=40.7128, lon=-74.0060`），时区设为 `America/New_York`。
  - 时间范围：2024-01-01 至 2024-12-31，小时频率。
  - 存储文件：`/data/tlc_hvfhv_2024_out/weather_hourly_2024.parquet`。
  - 字段说明：`ts_hour`（小时时间戳）、`temperature_2m`（2 米气温）、`precipitation`（降水量）、`wind_speed_10m`（10 米风速）、`weather_code`（天气现象编码）。
- 假期数据来源：`holidays`（Python 第三方库）生成的纽约州 2024 年公共假期日历。
  - 生成方式：在脚本中调用 `holidays` 生成 NY 州 2024 年日期集合，并进一步构造假期前一天与后一天标记。
  - 存储文件：`/data/tlc_hvfhv_2024_out/holidays_NY_2024.parquet`。
  - 字段说明：`date`（自然日）、`is_holiday`（是否假期）、`is_before_holiday`（假期前一日）、`is_after_holiday`（假期后一日）。

- Join 与对齐方式（用于复现特征构造过程）
  - 天气对齐：将行程请求时间 `request_datetime` 截断到整点得到 `ts_hour`（`dt.truncate("1h")`），再以 `ts_hour` 对 `weather_hourly_2024.parquet` 做左连接。
  - 假期对齐：将 `request_datetime` 转换为日期 `date`（`dt.date()`），再以 `date` 对 `holidays_NY_2024.parquet` 做左连接。
  - 缺失处理：天气连接失败时保留原样本，并用 `weather_missing=1` 标记该行天气未知；假期字段连接失败时将 `is_holiday`、`is_before_holiday`、`is_after_holiday` 填充为 0，表示非假期相关日。
  - 衍生字段：`is_precip` 由 `precipitation>0` 得到；`weather_missing` 由天气字段是否缺失得到。

1.2.3 原始数据字典

下表为原始数据中与本任务相关的核心字段。

表 1: 原始数据（HVFHV）字段字典

字段名	类型（建议）	中文说明
request_datetime	datetime	订单请求时间；用于构建小时、星期、月份、高峰、假期等特征
PULocationID	int	上车区域 Taxi Zone 编码（离散）
DOLocationID	int	下车区域 Taxi Zone 编码（离散）
trip_miles	float	行程里程（英里）；与费用/时长强相关
trip_time	int	行程时长（秒）；作为时长预测目标 y_{time}
base_passenger_fare	float	基础票价；用于合成 y_{cost}
tolls	float	过路费；用于合成 y_{cost}
bcf	float	附加费用之一；用于合成 y_{cost}
sales_tax	float	销售税；用于合成 y_{cost}
congestion_surcharge	float	拥堵附加费；用于合成 y_{cost}
airport_fee	float	机场附加费；用于合成 y_{cost}
shared_request_flag	str(Y/N)	是否拼车请求标记（Y/N）
wav_request_flag	str(Y/N)	是否无障碍车辆请求标记（Y/N）
access_a_ride_flag	str(Y/N)	是否 AAR（Access-A-Ride）相关请求标记（Y/N）
hvfhs_license_num	str	平台许可证编码（高基数/类别型；在本数据中可能实际取值很少）
dispatching_base_num	str	派单 base 编码（类别型）
originating_base_num	str	发起 base 编码（类别型，存在缺失）

1.2.4 特征工程后数据字典（训练用 features parquet）

下表为当前 `features_2024-XX.parquet` 中用于训练/评估的字段。

表 2: 特征工程后数据字段字典（中文说明）

字段名	类型（建议）	中文说明（含构造逻辑）
y_cost_no_tips	float32	目标变量：不含小费的开销；由票价与附加费求和得到
y_time	int32	目标变量：行程时长（秒）
hour	int8	请求时间的小时（0–23）
dow	int8	星期几（0–6）；由请求日期计算
is_weekend	int8	是否周末（星期六/日）
month	int8	请求月份（1–12）
is_peak	int8	是否高峰时段（例如 7–10 或 16–19，可按定义调整）
PULocationID	int32	上车区域编码（离散；直接输入模型）
DOLocationID	int32	下车区域编码（离散；直接输入模型）
same_zone	int8	是否同区出行（PU==DO）
route_id	int32	路线编码：`PU*1000 + DO`（离散，表征 OD 组合）
trip_miles	float32	行程里程（英里）
log1p_trip_miles	float32	里程对数变换：$\log(1 + \text{trip_miles})$
is_zero_miles	int8	新增：是否 0 里程（`trip_miles==0` $\rightarrow$ 1，否则 0）；用于捕捉异常/极短行程或记录口径问题
shared_request_flag	int8	拼车标记（Y/N $\rightarrow$ 0/1）
wav_request_flag	int8	无障碍车辆标记（Y/N $\rightarrow$ 0/1）
access_a_ride_flag	int8	AAR 标记（Y/N $\rightarrow$ 0/1）
originating_base_missing	int8	originating_base 缺失指示（缺失 =1，否则 =0）
hvfhs_code	int32	平台编码（对 hvfhs_license_num 进行全局 label encoding；UNK=0）
dispatch_code	int32	派单 base 编码（全局 label encoding；UNK=0）
origin_code	int32	发起 base 编码（全局 label encoding；UNK=0）
temperature_2m	float32	逐小时气温（摄氏度，按 `ts_hour` join）
precipitation	float32	逐小时降水量（毫米，按小时 join）
wind_speed_10m	float32	逐小时风速（10m，按小时 join）
weather_code	int32	天气现象编码（按小时 join；缺失可用 -1/0 处理）
is_precip	int8	是否有降水（precipitation>0）
weather_missing	int8	天气是否缺失（缺失 =1；用于提醒模型“未知天气”）
is_holiday	int8	是否法定假期（NY 州，按日期 join）
is_before_holiday	int8	是否假期前一天（按日期构造）
is_after_holiday	int8	是否假期后一天（按日期构造）

1.2.5 目标变量分布（长尾性与异常值）

HVFHV 行程开销与时长均呈现明显的长尾分布：大部分行程处于中低区间，但存在少量极端大额或超长时长记录（可能来自异常计费、极端交通状况或数据质量问题）。因此仅使用原始尺度直接建模会导致：

- 损失函数（如 MSE/RMSE）被极端值主导，模型更倾向于拟合尾部而牺牲主体区间；
- 目标变量的分布偏态较强，树模型虽然鲁棒但仍会受极端样本影响，线性基线模型更容易失真；
- 训练不稳定、泛化误差增大，且在抽样训练时极端值的随机出现会放大波动。

为缓解上述问题，本文对目标变量采用分位数截尾（winsorization）与对数变换（log1p）：

- **winsorization（分位数截尾）**：在训练集上估计 $q_{0.5\%}$ 与 $q_{99.5\%}$，将目标裁剪到 $[q_{0.5\%}, q_{99.5\%}]$ 区间内，以降低极端值对训练的影响；
- **log1p 变换**：对裁剪后的目标做 $y' = \log(1+y)$，压缩长尾、减小偏态，使模型更关注主体区间误差；预测时再通过 $\hat{y} = \exp(\hat{y}') - 1$ 还原到原始尺度进行评估。

表 3 给出了全年目标变量的关键分位数与最大值，用于直观展示长尾与极端值程度。

表 3: 目标变量分位数统计（全年汇总）

目标	p50	p90	p99	p99.5	max
y_cost_no_tips	22.95	61.96	142.03	170.92	2578.31
y_time	978.00	2308.00	4293.00	4914.00	55138.00

从表中可以看到，y_{cost} 在 p99.5 约为 170.92，但最大值达到 2578.31；y_{time} 在 p99.5 约为 4914 秒，但最大值达到 55138 秒（约 15.3 小时）。这类极端值虽然占比极低，但会显著影响回归训练与评估，因此后续建模阶段统一采用“分位数截尾 + log1p”的目标处理策略。

1.2.6 关键特征分布与合理性检查

- `trip_miles` 是否存在大量 0/极大值；
- `PULocationID/DOLocationID` 的类别覆盖；
- 天气字段是否按小时正确对齐（抽查若干日期/小时）；
- 假期标记是否与日历一致（抽查如 New Year’s Day、Thanksgiving 等）。

表 4: 里程分布、0 里程占比与 originating_base 缺失率（按月）

Month	Miles Mean	Miles Std	Miles Min	Miles Max	Miles=0 Cnt	Miles=0 Rate
Jan-24	4.8386	5.6987	0	417.6200	3286	0.0001671
Feb-24	4.9305	5.6821	0	289.2100	3082	0.0001592
Mar-24	4.9505	5.7489	0	271.3200	2954	0.0001388
Apr-24	5.0378	5.8342	0	279.3100	2508	0.0001271
May-24	5.1454	5.9473	0	326.3500	2510	0.0001212
Jun-24	5.1730	5.9903	0	363.5500	2655	0.0001319
Jul-24	5.1503	5.9777	0	350.8500	2479	0.0001292
Aug-24	5.1837	6.0234	0	488.6620	2594	0.0001356
Sep-24	5.2049	5.9693	0	469.4080	2991	0.0001557
Oct-24	5.1973	5.9636	0	455.5200	2913	0.0001454
Nov-24	5.0691	5.8626	0	339.2700	2869	0.0001435
Dec-24	5.1024	6.0377	0	555.2500	3218	0.0001527

注：Miles=0 Rate 为 0 里程样本占比；Miles 单位为英里。

表 5: 行程时长统计（按月）

Month	Time Mean	Time Std	Time Min	Time Max
Jan-24	1110.4697	772.2219	0	52060
Feb-24	1151.9185	800.6202	0	55138
Mar-24	1178.8639	840.0541	0	43235
Apr-24	1189.4508	843.2757	0	41560
May-24	1262.3402	931.5299	0	43234
Jun-24	1240.0238	903.1166	0	36156
Jul-24	1180.3203	829.3123	0	42478
Aug-24	1190.7289	847.6160	0	39352
Sep-24	1277.7795	931.6801	0	33105
Oct-24	1251.5100	887.6882	0	35091
Nov-24	1219.9194	878.6962	0	35714
Dec-24	1234.2684	922.9572	0	30150

注：Time 单位为秒；min/max 来自原始字段的极值（如需，可在数据清洗中进一步过滤异常极端值）。

表 6: 费用与小费统计（按月）

Month	Cost Mean	Cost Std	Cost Min	Cost Max	Tips Mean	Tips Max
Jan-24	28.9984	25.1094	-13.32	2017.97	1.0583	196.80
Feb-24	29.3749	25.0273	-114.02	1445.19	1.0701	188.00
Mar-24	30.8056	26.6251	-27.77	1606.40	1.1384	205.90
Apr-24	31.2571	27.4348	-19.37	1346.74	1.1771	266.63
May-24	32.7230	28.9532	-40.82	1934.59	1.2552	206.21
Jun-24	32.5109	28.9264	-13.92	1819.95	1.2191	204.47
Jul-24	30.9706	27.5171	-22.60	1463.36	1.1330	274.64
Aug-24	31.4837	27.8811	-19.86	1664.60	1.1138	299.07
Sep-24	32.9793	28.7987	-37.65	1528.56	1.1988	227.68
Oct-24	32.6230	28.6602	-12.17	1584.41	1.2508	237.10
Nov-24	31.7970	28.1960	-22.57	1934.04	1.1720	212.98
Dec-24	33.4281	31.1472	-23.63	2578.31	1.2857	336.21

注：Cost 为不含 tips 的费用合计；出现负值通常来自数据中的冲销/调整记录（可在清洗环节讨论是否截断或过滤）。

表 7: 0 里程子样本画像（按月）

Month	Miles0 Rows	Time Mean (Miles0)	Cost Mean (Miles0)	Tips Mean (Miles0)
Jan-24	3286	466.7426	13.8639	0.3351
Feb-24	3082	419.0039	13.5601	0.3009
Mar-24	2954	440.2146	15.0148	0.4119
Apr-24	2508	397.2783	14.7373	0.4303
May-24	2510	353.5709	13.9584	0.4274
Jun-24	2655	368.5066	15.0030	0.4245
Jul-24	2479	360.1948	13.9262	0.3113
Aug-24	2594	363.5085	14.3127	0.4177
Sep-24	2991	384.8840	15.1961	0.3445
Oct-24	2913	359.3780	15.0768	0.3847
Nov-24	2869	317.7170	14.5193	0.3358
Dec-24	3218	322.0562	16.0219	0.3325

表 8: 离散字段基数（unique counts，按月）

Month	PU uniq	DO uniq	hvfhs uniq	dispatch uniq	origin uniq	route uniq
Jan-24	262	261	2	2	7	58911
Feb-24	262	262	2	2	6	59135
Mar-24	261	263	2	2	8	59678
Apr-24	262	262	2	2	10	59658
May-24	262	261	2	2	8	60153
Jun-24	261	261	2	2	8	60593
Jul-24	262	262	2	2	7	60285
Aug-24	261	261	2	2	8	60345
Sep-24	262	262	2	2	8	60266
Oct-24	262	263	2	2	7	60210
Nov-24	262	262	2	2	8	59932
Dec-24	262	261	2	2	7	59849

1.3 训练数据构造与预处理流程

1.3.1 时间切分：避免未来信息泄露

为符合真实预测场景并避免未来信息泄露，本任务按月份进行时间序列切分：

$$\text{Train} = 2024/01\text{–}2024/09, \quad \text{Valid} = 2024/10, \quad \text{Test} = 2024/11\text{–}2024/12$$

并严格遵循如下实验规范：

- **目标截尾（clip）阈值**仅使用训练集数据估计；
- **模型超参数与特征选择**仅在验证集上比较与确定；
- **测试集仅做一次最终报告**，不反复查看测试集结果以避免隐性调参泄露。

1.3.2 哈希抽样策略：2% → 5%

由于 2024 年 HVFHV 数据规模极大，为在有限算力下完成多轮实验，本项目采用基于 `route_id` 的哈希抽样实现近似均匀随机采样：

```
hash(route_id) mod 100 < sample_pct
```

该策略的优点在于：抽样过程可复现（固定 seed），并且对 OD 组合（路线）具有一定的分层效果，有利于保持路线分布的代表性。

具体迭代策略如下：

- **早期迭代（2%）**：用于快速打通端到端 pipeline（特征构造 → 抽样 → 训练 → 评估）、排查 schema/缺失/类型问题，并进行特征增删对比与粗粒度调参；
- **最终训练（5%）**：将训练集抽样比例从 2% 提升至 5%，以减少抽样噪声、增强样本覆盖与结果稳定性；验证/测试仍使用 2% 以控制评估开销并保持对比一致性。

1.3.3 目标变量稳健变换：winsorization + log1p

由于 y_{cost} 与 y_{time} 呈长尾分布且存在极端异常值（见前文分位数统计），训练阶段对目标变量采用“分位数截尾 + log1p”的稳健变换：

- **winsorization（分位数截尾）**：在训练集抽样数据上估计 $q_{0.5\%}$ 与 $q_{99.5\%}$，并将目标裁剪到该区间内（具体阈值见前文）；
- **log1p 变换**：对裁剪后的目标做 $\tilde{y} = \log(1 + y)$，在对数尺度训练回归模型；推断时通过 $\hat{y} = \exp(\hat{\tilde{y}}) - 1$ 还原到原尺度，并在原尺度计算 RMSE/MAE。

1.3.4 缺失值与数据类型处理

为保证不同模型训练流程一致且可复现，本项目对缺失值与数据类型做统一处理：

- **缺失处理**：在导出 numpy 特征矩阵阶段，将特征中的 NaN 统一填充为 0（`np.nan_to_num`）；对于“缺失本身具有信息量”的字段，额外保留 missing indicator（例如 `weather_missing`、`originating_base_missing`）以提示模型该信息不可得；
- **树模型（XGBoost/LightGBM）**：可处理一定缺失情况，但为保持实现统一与训练稳定性，仍采用上述填充策略，并依赖 missing indicator 弥补信息损失；
- **Ridge**：不接受 NaN，必须完成缺失填充；同时对特征进行标准化（StandardScaler）以确保正则化项在不同量纲下具有可比性；
- **数据类型与内存优化**：在构造特征矩阵时将所有特征统一 cast 为 `float32`，显著降低内存占用并提升训练速度；标签在对数尺度训练，评估时还原至原尺度。

1.4 特征工程设计

1.4.1 建模时点选择：以 `request_datetime` 为预测时点（避免信息泄露）

本研究以 `request_datetime`（**乘客发起请求时刻**）作为预测时点进行建模，即模型在该时刻仅允许使用“请求当下即可获得”的信息。因此这里**不使用**在行程开始或结束后才能观测到的字段，例如 `pickup_datetime`、`dropoff_datetime`、`on_scene_datetime` 等，以避免信息泄露导致的虚高性能。

预测目标包含两部分：

- **乘客开销（不含 tips）**：由费用明细项构造

$$y_{\text{cost}} = \text{base_passenger_fare} + \text{tolls} + \text{bcf} + \text{sales_tax} + \text{congestion_surcharge} + \text{airport_fee},$$

  其中各费用项缺失值按 0 处理（表示未收取或不适用）。

- **乘坐时间**：直接使用 `trip_time`（秒）作为标签 y_{time}。

特征工程遵循“**简单、可解释、计算开销小**”的原则，并在确保不泄露未来信息的前提下，围绕价格与时长的主要决定因素构造特征。整体上分为四类：时间特征、空间路线特征、行程长度特征、平台与服务属性特征；并进一步融合天气与假期作为外部信息增强。

1.4.2 基础时空特征（时间周期性 + 空间差异）

时间特征：捕捉周期性需求与拥堵变化 从 `request_datetime` 提取：`hour`（小时）、`dow`（星期）、`is_weekend`（周末）、`month`（月份），用于刻画日内/周内/季节性周期。此外构造 `is_peak` 高峰时段标记（例如 7–10 或 16–19 点，按本文统一规则定义），作为粗粒度的拥堵 proxy，用于反映高峰期出行时间与费用的系统性上升。

空间与路线特征：刻画不同出行走廊的系统差异 直接使用 `PULocationID` 与 `DOLocationID` 表示上下车区域，并构造：

- `same_zone`：`PU==DO` 的指示变量（同区短途具有不同费用与耗时模式）；
- `route_id = PU*1000 + DO`：OD 路线编码，用于表征路线级差异（例如跨区通勤走廊与本地短途）。

该设计的动机是：在 NYC 城市结构下，不同 OD 组合往往对应不同道路条件、距离分布与拥堵特征，进而显著影响费用与耗时。

1.4.3 行程长度特征（距离对费用与耗时的核心解释力）

加入 `trip_miles` 与其非线性变换 `log1p_trip_miles`，用于刻画“距离”对费用与耗时的主导作用，并缓解里程长尾分布带来的训练不稳定。这里采用的假设是：在网约车下单时，平台通常能够基于导航给出预计路径与距离，因此 `trip_miles` 可以视为“请求时点可获得的近似信息”（或平台可估计量）。同时，为处理少量 `trip_miles=0` 的记录（可能来自极短途、取消/异常计费或记录误差），在特征中加入了指示变量 `is_zero_miles`，避免模型将其与常规短途混淆。

1.4.4 平台与订单属性特征（派单策略与服务类型差异）

平台/基地字段的编码：全局一致的 label encoding `hvfhs_license_num`、`dispatching_base_num`、`originating_base_num` 反映平台与派单基地差异，不同平台可能具有不同定价策略、补贴与派单逻辑，因此对预测目标具有重要解释力。本研究对三者进行 **全局一致**的 label encoding（跨月份统一映射），分别生成：`hvfhs_code`、`dispatch_code`、`origin_code`，并将缺失统一映射为特殊类别 `UNK`（编码为 0），以保证训练/验证/测试一致。

服务属性标记：请求时点即可观测 将 `shared_request_flag`、`wav_request_flag`、`access_a_ride_flag` 进行二值化（Y/N→0/1）。这些字段均为乘客请求时点可知的服务属性，会影响派车可得性、绕行概率与计价结构，从而影响费用与耗时。

缺失值处理：缺失本身也可能携带信息 `originating_base_num` 存在约 25% 的缺失。除将缺失填为 `UNK` 外，额外构造缺失指示变量 `originating_base_missing`（缺失 =1，否则 =0），以保留“缺失本身”对平台/派单机制的潜在解释信息。与之相对，`on_scene_datetime` 虽也存在较高缺失，但其语义属于司机到达信息，并非 request 时点可用，因此不纳入特征集合。

1.4.5 天气与假期特征（外部信息融合）

天气：按小时对齐的逐小时气象特征 将 `request_datetime` 截断到小时得到 `ts_hour`，与逐小时天气表按 `ts_hour` 左连接，加入 `temperature_2m`、`precipitation`、`wind_speed_10m`、`weather_code`。

并构造两个辅助变量：

- `is_precip`：若 `precipitation>0` 则为 1；
- `weather_missing`：天气 join 缺失指示（缺失 =1），用于提醒模型“此时天气未知”。

假期：按日期对齐的节假日效应 将 `request_datetime` 提取日期并与 NY 假期表按日期左连接，加入 `is_holiday`、`is_before_holiday`、`is_after_holiday`，用于捕捉假期与假期前后出行需求变化对费用与耗时的影响。

1.4.6 样本过滤与落盘（保证训练稳定与内存安全）

为减少明显异常对模型训练的干扰，对样本进行基本过滤：例如 `trip_time<=0`、`trip_miles<0`、`y_cost<0` 的记录将被剔除。最终将特征按月份写出为独立的 `features_2024-XX.parquet` 文件，从而支持后续按时间切分训练/验证/测试，并保证大规模数据处理的内存安全。

1.5 模型方法与训练策略

1.5.1 统一评估指标与还原方式

本项目对两类目标（不含小费的开销 y_{cost} 与行程时长 y_{time}）均采用一致的“训练空间–评估空间”方案：

- **在 log 空间训练：**先对目标做分位数截断（winsorization）得到 y'，再做 $\tilde{y} = \log(1 + y')$，模型在 $\tilde{y}$ 空间最小化 RMSE；
- **在原尺度评估：**推断时将预测值用 $\hat{y} = \exp(\hat{\tilde{y}}) - 1$（即 `expm1`）还原到原始尺度，并在 Valid/Test 上计算 RMSE 与 MAE。

该策略的动机是：原始目标存在显著长尾与极端值，直接在原尺度优化平方误差会被少量异常点主导，导致训练不稳定；而“截断 + 对数”能压缩长尾、提升收敛稳定性，同时仍可在原尺度给出可解释的误差指标。

1.5.2 训练阶段策略：时间切分、抽样训练与稳健目标变换

训练阶段采用“**按时间切分 + 抽样训练 + 目标稳健变换**”的流程，以兼顾计算可行性与评估的业务一致性：

- **按月份时间切分：**Train=2024/01–2024/09，Valid=2024/10，Test=2024/11–2024/12。该切分方式符合真实业务中的时间顺序，避免未来信息泄露；
- **抽样训练：**由于全年 HVFHV 数据规模达到数亿级别，全量加载训练会造成极大的时间与内存开销。因此在每个月数据上采用基于 `route_id` 的哈希过滤实现近似随机抽样：

```
hash(route_id) mod 100 < sample_pct
```

  该抽样方式在保持 OD 覆盖面与分布代表性的同时显著降低训练成本，使得多轮特征对比与参数迭代可以在可控时间内完成；

- **稳健目标变换：**仅在训练集抽样数据上估计分位数阈值（如 $q_{0.005}, q_{0.995}$），对目标进行 winsorization 截断，再做 `log1p` 变换；推断时再 `expm1` 还原并在原尺度评估。

1.5.3 XGBoost（hist）

选择理由

- 对非线性关系与特征交互具有较强建模能力，适合本任务“时空 + 平台属性 + 天气假期”混合特征；
- `tree_method=hist` 在 CPU 上对大规模样本训练更高效；
- 可输出特征重要性，便于后续解释各类特征（时间、空间、里程、平台属性、天气与假期）对费用与时长预测的贡献。

训练配置

- boosting rounds：2000（`num_boost_round=2000`）
- early stopping：120（`early_stopping_rounds=120`）
- tree method：`hist`
- 关键参数：`learning_rate=0.08, max_depth=8, min_child_weight=10, subsample=0.9, colsample_bytree=0.9, max_bin=256, lambda=1.0, alpha=0.0`

1.5.4 LightGBM（原版 → 更快更稳版）

选择理由

- LightGBM 在 CPU 上通常训练速度快，适合大规模 tabular 数据；
- 对离散/稀疏特征适配良好。本项目将离散特征统一数值化输入（Taxi Zone、route、平台编码等），可直接使用；
- 结合 early stopping，能够在保证泛化性能的同时控制训练迭代成本。

抽样比例设置（两次训练一致） 为保证对比公平性，两次 LightGBM 训练均采用相同的数据抽样比例设置：

`SAMPLE_TRAIN=5%,  SAMPLE_VALID=2%,  SAMPLE_TEST=2%.`

其中训练集提高到 5% 以增强样本代表性与稳定性；验证与测试保持 2% 以控制评估成本并减少重复计算压力。

参数演进说明（原版 → 更快更稳版） LightGBM 的参数经历两次主要版本迭代，核心思路是：在保持整体采样比例不变的前提下，通过控制树复杂度、提升学习率来减少无效迭代，并增强训练稳定性。

- **第一次训练（原版配置）：**强调较小学习率与较大的树容量，以获得更强的拟合能力。

- **第二次训练（更快更稳版）：**提高学习率、降低叶子数并提高叶子最小样本数，以降低模型复杂度、减少过拟合风险，同时减少达到较优点所需的 boosting 轮数。

表 9: LightGBM 两次训练参数对比（原版 vs 更快更稳版）

参数	原版（第一次训练）	更快更稳版（第二次训练）
objective / metric	regression / rmse	regression / rmse
learning_rate	0.06	0.10
num_leaves	128	96
min_data_in_leaf	80	150
feature_fraction	0.9	0.9
bagging_fraction	0.9	0.9
bagging_freq	1	1
max_bin	255	255
seed	42	42

为何“更快更稳” 第二版参数组合的直观效果可以概括为：

- **更快：**提高 `learning_rate` 往往能用更少 boosting 轮数达到相近误差水平（配合 early stopping 可减少无效训练）；
- **更稳：**降低 `num_leaves` 并提高 `min_data_in_leaf` 会限制树的细碎分裂，减少对噪声与极端样本的过拟合，使训练与验证曲线更平滑；
- **更可控：**在大样本场景中，更强的正则化（更大的叶子最小样本数）有助于提高跨月泛化稳定性。

1.5.5 Ridge 回归（线性基线）

选择理由 Ridge 作为可解释线性基线模型，用于衡量“仅依赖线性关系”的性能下限，并帮助解释 GBDT 非线性模型的增益来源：

- 训练速度快、内存占用可控；
- 通过系数可解释各特征的线性方向性；
- 与树模型对比可说明非线性与交互项的重要性。

抽样口径（与树模型区分） 为保证 Ridge 的训练成本可控并与“快速基线”的定位一致，Ridge 模型采用统一的 2% 哈希抽样口径：

`SAMPLE_TRAIN=2%,  SAMPLE_VALID=2%,  SAMPLE_TEST=2%.`

该设置与树模型的最终训练（Train=5%，Valid/Test=2%）不同，属于更严格计算预算下的线性基线对比。

训练配置（与树模型对齐）

- 特征标准化：`StandardScaler(with_mean=True, with_std=True)`；
- 正则项：`Ridge(alpha=5.0)`（固定超参数，作为同口径基线）；
- 目标处理与评估方式与树模型一致：训练在 $\tilde{y} = \log(1 + y')$ 空间，推断用 `expm1` 还原到原尺度并计算 RMSE/MAE。

1.5.6 抽样比例与数据规模（2% vs 5%）

- 最终采用：**Train 5% / Valid 2% / Test 2%**
- 样本量（cost 与 time 一致）：

$$n_{\text{train}} = 8{,}481{,}241, \quad n_{\text{valid}} = 382{,}503, \quad n_{\text{test}} = 794{,}908$$

- 对比基线（Ridge 默认同口径抽样）：**Train 2% / Valid 2% / Test 2%**，对应

$$n_{\text{train}} = 3{,}450{,}955, \quad n_{\text{valid}} = 382{,}503, \quad n_{\text{test}} = 794{,}908$$

1.5.7 结果表

表 10: LightGBM 参数改进前后对比（Cost，原尺度评估）

版本	Valid RMSE	Valid MAE	Test RMSE	Test MAE
初版（lr=0.06, leaves=128, min_leaf=80）	9.1035	4.5273	10.1857	5.1485
更快更稳版（lr=0.10, leaves=96, min_leaf=150）	9.1285	4.5425	10.1702	5.1739
变化（更快更稳版 − 初版）	0.0250	0.0152	-0.0155	0.0254

表 11: LightGBM 参数改进前后对比（Time，原尺度评估）

版本	Valid RMSE	Valid MAE	Test RMSE	Test MAE
初版（lr=0.06, leaves=128, min_leaf=80）	301.7096	185.0628	311.2658	188.4726
更快更稳版（lr=0.10, leaves=96, min_leaf=150）	302.2104	185.0550	312.4599	188.8477
变化（更快更稳版 − 初版）	0.5008	-0.0078	1.1941	0.3751

两次训练使用相同的数据切分、抽样比例与 winsorization 截断阈值，因此差异主要来自 LightGBM 参数调整。

“更快更稳版”整体误差与初版非常接近：Cost 在 Test RMSE 上略有下降，但其余指标轻微波动；Time 指标整体略有上升（差距也不大）。

表 12: 费用预测（Cost，不含 tips）在原尺度上的结果对比

模型	Valid RMSE	Valid MAE	Test RMSE	Test MAE
XGBoost（log1p + clip）	9.1841	4.5701	10.2823	5.6390
LightGBM 初版	9.1035	4.5273	10.1857	5.1485
LightGBM 更快更稳版	9.1285	4.5425	10.1702	5.1739
Ridge（alpha=5.0，train=2%）	11.8278	6.9670	13.3622	7.5705

表 13: 时长预测（Time）在原尺度上的结果对比

模型	Valid RMSE	Valid MAE	Test RMSE	Test MAE
XGBoost（log1p + clip）	304.0581	186.4621	313.6260	193.2240
LightGBM 初版	301.7096	185.0628	311.2658	188.4726
LightGBM 更快更稳版	302.2104	185.0550	312.4599	188.8477
Ridge（alpha=5.0，train=2%）	475.7084	299.4974	522.4340	322.3152

1.5.8 结果解读

在相同时间切分与抽样口径下，两类 GBDT（XGBoost、LightGBM）显著优于 Ridge 线性基线。以 Cost 为例，Ridge 的 Test RMSE 为 13.36，而树模型约为 10.17–10.28；Time 的差距更明显（Ridge Test RMSE 522.43 vs 树模型约 312–314）。这表明费用与时长与特征之间存在明显的非线性关系与交互效应（例如时间段与路线、天气与高峰叠加等），线性模型难以充分拟合。

在两种树模型之间，LightGBM（更快更稳版）整体略优于 XGBoost：Cost 的 Valid/Test RMSE 分别降低约 0.056/0.112，且 Test MAE 下降更明显（5.17 vs 5.64），说明其在费用任务上对典型误差与尾部误差都有一定改善；Time 任务上 LightGBM 同样略优（Test RMSE 312.46 vs 313.63，Test MAE 188.85 vs 193.22），表现为更稳定的平均绝对误差。总体来看，在本项目的特征集合与训练设置下，LightGBM 是更合适的主力模型，而 XGBoost 作为强基线提供了接近的性能对照，Ridge 则用于量化“仅线性关系”下的下限表现。

1.6 小结与改进方向

1.6.1 小结

- 本研究以 `request_datetime`（乘客发起请求时刻）作为预测时点，构造不含小费的费用目标

  $$y_{\text{cost}} = \text{base_passenger_fare} + \text{tolls} + \text{bcf} + \text{sales_tax} + \text{congestion_surcharge} + \text{airport_fee},$$

  并以 `trip_time`（秒）作为时长目标 y_{time}，避免使用行程开始/结束后才能观测到的字段以防信息泄露。

- 采用严格按月份的时间序列切分（Train=1–9 月，Valid=10 月，Test=11–12 月），并确保 clip 阈值仅由训练期样本估计、超参数仅由验证集调优、测试集只报告一次，评估流程符合真实上线场景。

- 面对 y_{cost} 与 y_{time} 的明显长尾与极端值问题，训练阶段对目标做 winsorization（0.5%–99.5% 分位截断）并进行 `log1p` 变换，在不改变整体趋势的前提下减弱异常值对 RMSE 的主导作用，从而提高训练稳定性与收敛效率。
- 在相同特征集（时间、空间/路线、里程、平台与服务属性、天气与假期）下，改进版 LightGBM 与 XGBoost 的效果整体接近；线性基线 Ridge 明显落后，说明费用/时长与特征之间存在显著非线性与交互结构，GBDT 更适合该任务。
- 训练抽样采用基于 `route_id` 的哈希过滤实现近似随机抽样；最终使用 **Train 5% / Valid 2% / Test 2%**（训练期样本量 8,481,241），相较早期更小抽样，结果更稳定且仍可在可接受的计算成本下完成多轮实验。

1.6.2 改进方向

由于电脑性能和服务器等因素，本研究没有进行复杂的调参，此外仍有较多可以考虑改进的地方：

- **更精细的时间交互特征：**加入 `hour$times $dow`、`is_peak$times $precipitation`、`month$times $is_weekend` 等交互项，刻画工作日/周末不同时段的系统性差异与天气对高峰拥堵的放大效应。
- **路线统计先验（严格避免泄露）：**仅使用训练期（1–9 月）对 `route_id` 聚合历史均值/分位数（如平均费用、平均时长、p90 时长等），并将这些统计量作为先验特征注入验证/测试；可进一步采用 “leave-one-month-out” 的方式生成更稳健的先验以降低过拟合。
- **空间距离与邻接信息：**若可获得 Taxi zone 的中心点坐标或 shapefile，可构造 PU/DO 的欧式距离、跨区距离分桶、相邻区指示等空间特征；也可引入 “跨河/跨桥” 等地理分割的粗粒度特征提升可解释性。
- **更贴近拥堵机理的 proxy：**在不引入泄露字段的前提下，可从历史训练期估计 “小时 × 区域” 的平均速度/平均时长作为拥堵先验，或构造 “需求强度”（如同小时同区请求量的历史均值）来增强对拥堵变化的刻画。
- **LightGBM 正则与采样进一步调优：**在现有 “更快更稳” 参数基础上，可继续尝试更强的 `lambda_l1/l2`、更小的 `feature_fraction`、更大的 `min_data_in_leaf`，以及 `bagging_fraction` 的微调以提升泛化并缩小 train-valid gap。
- **数据质量与异常处理：**进一步系统化排查异常（如 `trip_time=0`、极端里程/费用、疑似退费导致的负费用记录），并将异常原因分组统计写入附录，为后续清洗策略提供依据。

2 第二部分：构建对城市拥堵程度进行建模的模型

2.1 问题定义与建模策略

城市拥堵是一个复杂的多维时空现象。对于网约车平台而言，拥堵不仅意味着“车速慢”，其背后的成因（是由于出行需求过大，还是由于恶劣天气影响）决定了不同的运营策略。

考虑到数据的海量特征，若仅预测具体的行驶速度，难以揭示交通状态的内在结构。因此，本研究提出了一种**“聚类定义 + 分类预测” (Clustering-then-Classification)** 的两阶段混合建模策略：

- **阶段一（模式发现）：**利用无监督学习（K-Means），根据车速、流量和天气特征，自动识别并定义城市中存在的典型拥堵模式。
- **阶段二（状态预测）：**基于阶段一得到的模式标签，构建监督学习模型（Random Forest），利用时间、地点和天气预报信息，预测特定区域在未来时刻的交通状态。

2.2 数据处理与特征工程

鉴于原始 HVFHV 数据集达到数亿行级别，且单次行程容易受个体驾驶习惯、红绿灯随机性等噪声影响，直接进行微观建模既不具备计算可行性，也难以捕捉宏观规律。因此，本研究设计了“清洗-聚合-融合”的数据处理流水线。

2.2.1 数据清洗与预处理

在聚合之前，为保证统计指标的稳健性，对原始行程记录进行了如下清洗：

- **异常值过滤**：剔除行驶时间小于 60 秒或里程小于 0.1 英里的无效行程；剔除平均速度超过 80 mph（数据错误）或低于 0.5 mph（可能为长时间停车等待）的异常记录。
- **时空对齐**：将所有时间戳统一截断至小时粒度（Floor to Hour），使用数据集提供的 NYC Taxi Zone ID (PULocationID) 作为空间聚合单元，将连续的地理空间离散化为 263 个标准区域。

2.2.2 时空聚合与特征构造

本研究采用**“日期-小时-上车区域”**（Date-Hour-PULocationID）作为基本时空单元进行聚合。对于每一个单元，构造了包含交通流属性、时间属性及环境属性的多维特征向量。详细的特征定义如表 14 所示。

表 14: 城市拥堵建模特征体系详细定义表

特征类别	特征名称	构造逻辑与物理含义
交通状态	**平均速度** (avg_speed)	**核心拥堵指标**。计算公式为 $\bar{v} = \frac{\sum \text{Miles}}{\sum \text{Time}}$。该值越低，代表该区域在当前小时内交通流速越慢。
	订单密度 (trip_count)	**核心需求指标**。统计该时空单元内的总订单数。用于区分由于“车多”导致的拥堵和由于“路况差”导致的拥堵。
	速度波动性 (std_speed)	**稳定性指标**。计算单次行程速度的标准差。高波动性通常意味着交通流处于“走走停停”的不稳定状态。
外部环境	**降雨量** (precipitation)	**气象干扰**。单位 mm，源自关联的逐小时气象数据。用于捕捉湿滑路面对通行能力的非线性削减作用。
	气温 (temperature_2m)	**季节性代理**。单位 °C。气温变化隐含了季节（冬夏）与昼夜（冷暖）的深层规律。
时序属性	**小时** (hour)	取值 0-23。用于捕捉早晚高峰、深夜畅通等日内通勤规律。
	是否周末 (is_weekend)	二值变量（0/1）。用于区分工作日通勤模式与周末休闲模式。
	节假日 (is_holiday)	二值变量（0/1）。标记法定节假日，此时通常伴随反常的交通流量分布。

2.2.3 标准化处理

由于不同特征的物理量纲差异巨大（例如：平均速度约为 15 mph，而订单密度可达 2000+），为了防止数值较大的特征主导 K-Means 算法的欧氏距离计算，在建模前对所有连续型特征（速度、流量、气温等）进行了 Z-Score 标准化处理：

$$x' = \frac{x - \mu}{\sigma} \tag{1}$$

其中 μ 为样本均值，σ 为样本标准差。

2.3 第一阶段：基于聚类的拥堵模式发现

利用 K-Means 算法（设定 $K = 4$）对 2024 年全年的时空聚合数据进行聚类，成功识别出四种显著的交通模式，统计结果如表 15 所示。

表 15: 2024 年城市拥堵模式聚类结果统计

Cluster	Speed (mph)	Count	Rain (mm)	Hour	模式定义
0	14.61	127.69	**5.29**	13.61	**恶劣天气缓行**
1	**20.33**	**44.60**	0.07	7.95	**自由流/畅通**
2	**12.78**	**187.23**	0.10	13.47	**高需求拥堵**
3	13.42	102.87	0.06	13.31	**常态通勤**

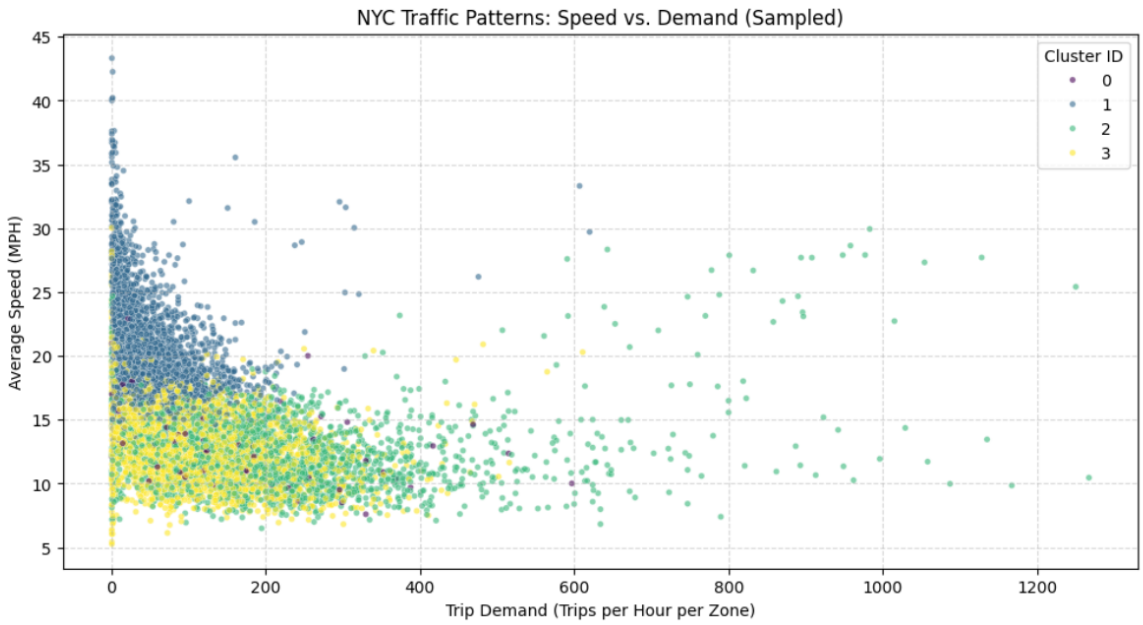

图 1: NYC Traffic Patterns: Speed vs. Demand (Sampled)

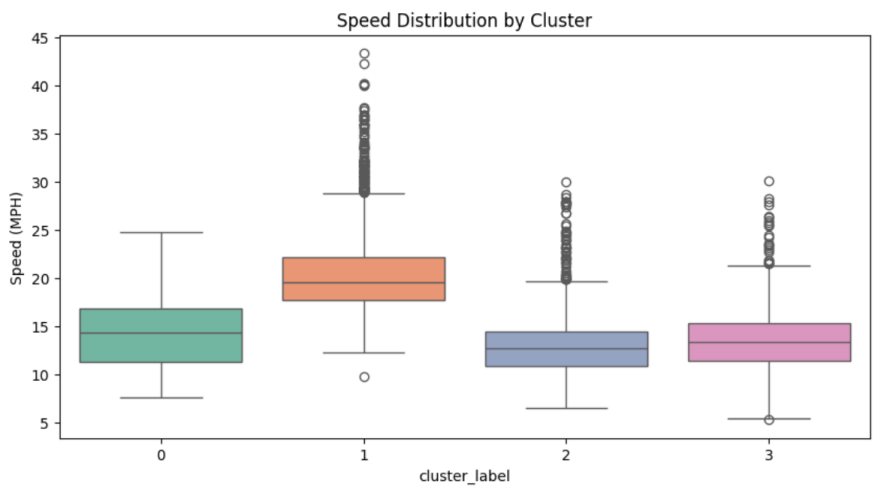

图 2: Speed Distribution by cluster

2.3.1 结果解读

- **Cluster 0 (恶劣天气缓行)**：最显著特征是极高的降雨量（5.29mm）。此时车速较低并非因为车多，而是因为路面湿滑导致通行效率下降。
- **Cluster 1 (自由流)**：拥有最高的车速和最低的订单量，通常出现在清晨或深夜，道路处于非饱和状态。
- **Cluster 2 (高需求拥堵)**：最低的车速对应最高的订单密度（Cluster 1 的 4 倍以上）。这是典型的由供需失衡导致的严重拥堵。
- **Cluster 3 (常态通勤)**：各项指标居中，代表了大多数工作日白天的标准路况。

2.4 第二阶段：拥堵状态预测模型

为了验证挖掘出的交通模式是否具有可预测性，并构建具有实战意义的拥堵预警系统，本研究将第一阶段得到的聚类结果（Cluster Labels）作为预测目标 Y，构建了监督学习分类模型。

2.4.1 模型构建与实验设置

1. **预测目标 (Y)**：即上一阶段 K-Means 输出的聚类标签（0, 1, 2, 3）。这代表了该区域在该小时将处于何种交通状态（如恶劣天气缓行、高需求拥堵等）。
2. **输入特征 (X)**：特征选择严格遵循“事前可知”原则，即仅使用在预测时刻能够获取的信息，严禁使用当且仅当行程发生后才知道的“未来数据”（如实际平均速度、实际订单量）。最终入模特征如下：
   - **环境预报特征**：气温 (`temperature_2m`)、降雨量 (`precipitation`)。
   - **时序特征**：小时 (`hour`)、星期 (`day_of_week`)、是否周末 (`is_weekend`)、是否节假日 (`is_holiday`)。
   - **空间特征**：区域编码 (`PULocationID`)，用于捕捉不同街区的固有拥堵概率。
3. **算法与划分**：采用 **随机森林分类器 (Random Forest Classifier)**，因其具有良好的抗过拟合能力和特征解释性。数据集按 8:2 的比例划分为训练集与测试集。

2.4.2 实验结果与评估

模型在测试集（样本量 $N = 443,088$）上的整体加权平均准确率（Weighted Accuracy）达到了 **79%**。具体的分类性能如表 16 所示。

表 16: 拥堵模式预测分类评估报告

模式类型 (Cluster)	Precision	Recall	F1-Score	Support
0: 恶劣天气缓行	**0.98**	**0.98**	**0.98**	6,195
1: 自由流/畅通	0.77	0.70	0.74	148,153
2: 高需求拥堵	0.76	0.77	0.76	128,276
3: 常态通勤	0.82	0.88	0.85	160,464
总体准确率 (Accuracy)			**0.79**	443,088
加权平均 (Weighted Avg)	0.79	0.79	0.79	443,088

结果分析：

- **恶劣天气模式 (Cluster 0) 极易预测**：其 F1-Score 高达 0.98，说明只要降雨特征出现，模型几乎能 100% 准确预警。这对于平台在雨雪天提前调整 ETA 具有极高价值。
- **拥堵与常态的区分**：Cluster 1（畅通）、Cluster 2（拥堵）和 Cluster 3（常态）的 F1 分数在 0.74-0.85 之间。这说明虽然存在一定的分类模糊边界（例如“常态”与“微堵”之间的界限），但模型依然能以约 80% 的概率准确判断未来是否会发生严重拥堵。

2.4.3 拥堵成因与特征重要性分析

为了解构模型是如何做出预测的，这里提取了随机森林的特征重要性（Feature Importance），结果如表 17 所示。

表 17: 拥堵模式预测模型的特征重要性

特征名	重要性权重	业务解读
temperature_2m	0.5137	**季节性/日温差**：气温隐含了季节与昼夜深层规律
hour	0.3417	**通勤规律**：早晚高峰是拥堵的核心驱动力
PULocationID	0.0615	**空间差异**：不同区域的基础路况不同
precipitation	0.0585	**天气触发器**：是识别 Cluster 0 的决定性特征
day_of_week	0.0139	工作日与周末的差异
is_weekend	0.0068	辅助时间特征
is_holiday	0.0038	节假日影响相对局部

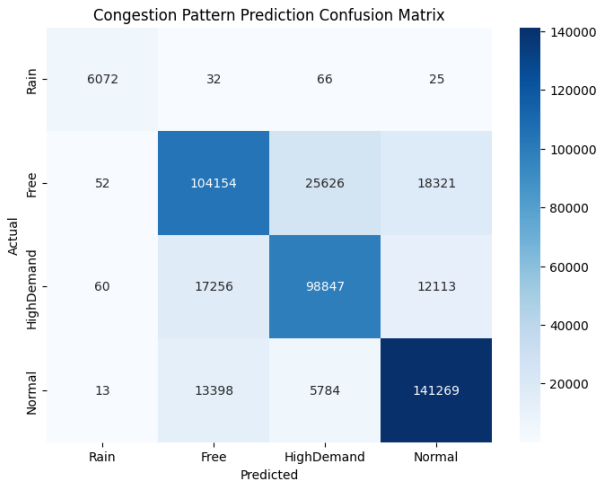

图 3: Confusion Matrix

关键发现：

1. **气温 (Temperature) 的主导作用 (51.4%)**：这是一个非常有趣的发现。气温在模型中不仅代表冷暖，更是一个强有力的“时间代理变量”。它同时捕捉了季节性变化（冬夏路况差异）和日内变化（白天热晚上冷），因此成为区分不同交通模态的最强信号。
2. **小时 (Hour) 的核心地位 (34.2%)**：时间是预测交通最硬的指标。结合气温和小时，模型便能解释 85% 以上的拥堵变化。
3. **降雨 (Precipitation) 的特异性**：虽然全局重要性仅为 5.8%，但它在特定场景（Cluster 0）下具有“一票否决权”。

2.5 结论与业务价值

本研究通过两阶段建模，不仅量化了拥堵，更区分了拥堵的成因：

- 针对 **Cluster 2 (高需求拥堵)**：建议平台触发动态定价（Surge Pricing）以调节供需，并调度运力。
- 针对 **Cluster 0 (天气缓行)**：增加运力无效，建议平台优化 ETA 算法，增加因天气导致的不确定性缓冲时间。

3 总结与展望

本项目基于 2024 年 NYC TLC HVFHV 海量网约车订单数据，通过构建两套不同的机器学习工作流，成功解决了微观层面的行程属性预测与宏观层面的城市拥堵建模问题。主要结论如下：

1. **在行程属性预测方面**：针对乘客打车开销与乘坐时间的回归任务，本项目实施了“按月时间切分 + 哈希抽样 + 目标稳健变换”的数据策略，有效克服了亿级数据规模带来的计算挑战。模型对比实验表明，梯度提升树模型在捕捉复杂的时空特征交互方面具有决定性优势。具体而言，**LightGBM 与 XGBoost 的预测精度相当且均远优于线性 Ridge 基线**；其中 LightGBM 在训练效率与稳定性上略优于 XGBoost，更适合大规模工业部署。

2. **在城市拥堵建模方面**：针对拥堵成因与状态预测任务，本项目提出了“聚类定义 + 分类预测”的两阶段综合方案。首先，利用 K-Means 无监督算法成功识别并定义了“恶劣天气缓行”、“高需求拥堵”等四类典型交通模式，揭示了拥堵背后的不同驱动机理；随后，构建了 Random Forest 分类器，证明了仅依据事前的时间与气象预报数据，即可实现对特定区域未来拥堵状态的有效预警（加权准确率达 79%）。

A 附录：关键代码实现

A.1 第一部分：特征工程与模型训练 (LightGBM)

```python
import lightgbm as lgb

# 最终优化参数
params = {
    "objective": "regression",
    "metric": "rmse",
    "learning_rate": 0.10,
    "num_leaves": 96,
    "min_data_in_leaf": 150,
    "feature_fraction": 0.9,
    "bagging_fraction": 0.9,
    "seed": 42
}

# 训练模型
model = lgb.train(
    params,
    train_set=lgb_train,
    valid_sets=[lgb_valid],
    num_boost_round=2000,
    callbacks=[lgb.early_stopping(stopping_rounds=100)]
)
```

Listing 1: LightGBM 训练代码片段

A.2 第二部分：聚类与分类建模

```python
from sklearn.cluster import KMeans
from sklearn.ensemble import RandomForestClassifier

# --- 阶段一：无监督聚类 ---
# df_agg 为聚合后的时空特征数据
kmeans = KMeans(n_clusters=4, random_state=42)
df_agg['cluster_label'] = kmeans.fit_predict(X_scaled)

# --- 阶段二：监督分类预测 ---
# 目标：根据时间、地点、天气，预测下一小时的交通模式
X = df_agg[['hour', 'day_of_week', 'PULocationID', 'precipitation', 'temperature_2m']]
y = df_agg['cluster_label']

# 划分数据集并训练
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
rf = RandomForestClassifier(n_estimators=100, max_depth=12, n_jobs=-1)
rf.fit(X_train, y_train)

```

```python
# 输出评估
print("Accuracy:", rf.score(X_test, y_test))
print("Feature Importances:", rf.feature_importances_)
```

Listing 2: K-Means 与 RandomForest 两阶段建模